Web Application Requirements: Customer Gift Payment Platform

1. Overview

A web application (backend in Go) that allows customers to:

- Register and manage an account
- Save payment card details for future use
- View their saved card details
- Use a saved card to pay for gifts

Important note on card data before you build this: storing and displaying full card numbers (PAN) and CVV yourself puts you inside PCI-DSS scope, which is a serious, expensive compliance burden (annual audits, network segmentation, encryption key management, etc.), and most card networks/acquirers won't let a non-certified merchant store raw PANs at all. The industry-standard approach is to **never touch or store the raw card number** — instead use a payment processor's tokenization (Stripe, Braintree, Adyen, etc.). The processor's hosted field/SDK collects the card, returns you a token + a masked representation (e.g. `**** **** **** 4242`), brand, expiry), and you store only that token. "Viewing card info" then means viewing the masked/token representation, not the real number. The requirements below are written with this safer architecture in mind — I've noted where I substituted "raw storage" with "tokenized storage" so you can compare against what you originally had in mind.

---

2. Functional Requirements

2.1 Account Management

- FR-1: A visitor can register with email, password, and basic profile info (name, phone – optional).
- FR-2: Passwords are hashed (bcrypt/argon2) — never stored in plaintext.
- FR-3: A registered customer can log in and receive a session token (JWT or server-side session).
- FR-4: A customer can view/update their profile.
- FR-5: A customer can log out; sessions/tokens expire after a configurable period.
- FR-6: Email verification and password-reset flow (recommended, not strictly mandatory for MVP).

2.2 Card Management

- FR-7: A customer can add a payment card. The raw card number/CVV is sent directly from the browser to the payment processor's SDK (client-side), never to your Go backend. Your backend only receives a **token** and **display metadata** (last 4 digits, brand, expiry month/year, cardholder name).

- FR-8: A customer can view their saved card(s) as masked info only (e.g. "Visa ending in 4242, exp 08/29"). Full PAN/CVV are never retrievable after entry — this matches how every major platform (Amazon, Stripe Checkout, etc.) works.
- FR-9: A customer can delete a saved card (revokes the token via the processor's API).
- FR-10: A customer can set a default card.

2.3 Gift Payment

- FR-11: A customer can browse/select a gift (simple catalog: name, price, image, description).
- FR-12: A customer can pay for a gift using a saved card token.
- FR-13: The backend calls the payment processor's charge/PaymentIntent API using the stored token — actual card data flows only between the customer's browser and the processor.
- FR-14: The system records a transaction history (gift, amount, status, timestamp, masked card used).
- FR-15: A customer can view their order/payment history.

2.4 Admin (optional but typical)

- FR-16: An admin can view customer list, transactions, and gift catalog.
- FR-17: An admin can manage the gift catalog (CRUD).

---

3. Non-Functional Requirements

- NFR-1 (Security): All traffic over HTTPS/TLS 1.2+. No card PAN/CVV ever logged, stored, or transmitted to your own servers.
- NFR-2 (Compliance): Architecture should keep you at PCI-DSS **SAQ-A** level (lowest burden) by using a hosted/iframe/SDK field from the processor, rather than SAQ-D (full scope) from handling raw card data yourself.
- NFR-3 (Auth security): Passwords hashed with bcrypt/argon2 + per-user salt; rate-limit login attempts; JWT with short expiry + refresh tokens, or secure server-side sessions.
- NFR-4 (Data protection): Sensitive fields (tokens, personal info) encrypted at rest where applicable; principle of least privilege on DB access.
- NFR-5 (Auditability): Log all payment attempts (success/failure) and admin actions, without logging sensitive data.
- NFR-6 (Availability): Basic health checks; graceful error handling on processor API failures.
- NFR-7 (Scalability): Stateless API servers behind a load balancer; DB connection pooling.
- NFR-8 (Input validation): Validate all inputs server-side (email format, password strength, amounts, etc.) regardless of client-side checks.

---

4. Suggested Tech Stack (Go)

Layer	Suggestion
HTTP framework	`net/http` + `chi`, or `gin`, or `echo`
ORM / DB access	`sqlc`, `gorm`, or plain `database/sql`
Database	PostgreSQL
Auth	`golang-jwt/jwt` for JWT, `golang.org/x/crypto/bcrypt` for hashing
Payment processor	Stripe Go SDK (`stripe-go`) — has strong tokenization + PaymentIntents support
Migrations	`golang-migrate`
Config	`viper` or plain env vars
Testing	standard `testing` + `testify`

---

5. Data Model (high level)

users `id, email (unique), password_hash, name, phone, created_at, updated_at`

payment_methods `id, user_id (FK), processor_token, brand, last4, exp_month, exp_year, is_default, created_at`

gifts `id, name, description, price_cents, currency, image_url, active`

orders `id, user_id (FK), gift_id (FK), payment_method_id (FK), amount_cents, currency, status (pending/succeeded/failed), processor_charge_id, created_at`

(Note: no `card_number` or `cvv` column anywhere — by design.)

---

6. Core API Endpoints (sketch)

```
POST   /api/auth/register
POST   /api/auth/login
POST   /api/auth/logout
GET    /api/me
PATCH  /api/me

POST   /api/payment-methods        # save token+metadata from processor
GET    /api/payment-methods        # list masked cards
DELETE /api/payment-methods/{id}

GET    /api/gifts
GET    /api/gifts/{id}

POST   /api/orders                 # {gift_id, payment_method_id} -> charges
via processor
GET    /api/orders                 # order history
GET    /api/orders/{id}
```

---

7. Open Questions to Resolve Before Building

1. Which payment processor will you integrate with (Stripe, Braintree, local acquirer)? This affects the client-side SDK and tokenization flow.
2. Do you need recurring/subscription payments, or one-off gift purchases only?
3. Do you need multi-currency support?
4. What's your expected scale (helps decide DB choice, caching needs)?
5. Do you need an admin dashboard as part of this app, or is that a separate internal tool?